National Teachers College
629 J Nepomuceno, Quiapo,
Manila, 1001 Metro Manila

Software Maintenance Plan

Aguilon, Diana Shane V.

Bantug, Rhaily C.

Jugueta, Kurt Clein

Tapangan, Sean Lawrence D.

Tolentino, John Michael D.

Software Engineering

Submitted to: Ms. Leyna V. Laure

Date:

National Teachers College
629 J Nepomuceno, Quiapo,
Manila, 1001 Metro Manila

Introduction

This plan explains how our group will take care of our cafe ordering website after we put it online. The website lets cafe staff pick drinks and pastries, place an order, and see the order queue. Software still needs care after it is finished. Bugs can show up, and we may want to add new things later. This plan tells how we will handle these changes so the website keeps working for the cafe staff.

Bug Tracking

We will use GitHub Issues to keep track of bugs. When someone finds a problem, we make a new issue and write what happened, the steps to see the problem, and what should have happened. We add a screenshot if we can.

Each issue gets a label: bug, improvement, or question. We also mark how serious it is.

- **High:** Customer cannot order or the site does not open. We will fix it on the same day.
- **Medium:** Something works wrong, like a wrong total. We will fix it within 3 days.
- **Low:** Small things like a typo or spacing. We will fix it in the next update.

Every week, one member checks the new issues and gives each one to a teammate. When the fix works on the live site, that member closes the issue. We also open the website once a week to look for problems.

Patching Process

A patch is a small fix. We use the same steps every time so a fix will now break something else.

- Read the issue and try to see the bug on our own computer.
- Make a new branch for the fix, like fix/cart-total.
- Change the code and test it. We check the menu page, the cart, and the order queue.
- Commit the change with a clear message, like “fix: correct cart total (closes #3)”.
- Open a pull request. One teammate checks it and approves it.
- Merge it into the main branch. Vercel then updates the live website by itself.
- Open the live link to check that the fix works, then close the issue.

For a serious bug we work faster, but a teammate still has to check the change.

National Teachers College
629 J Nepomuceno, Quiapo,
Manila, 1001 Metro Manila

Version Labeling Using SemVer

We label each version with three numbers: Major, Minor, Patch, for example 1.0.0 to 1.0.1.

- Patch goes up when we fix a bug. Example: 1.0.0 to 1.0.1.
- Minor goes up when we add a new feature that does not break the old ones. Example: 1.0.1 to 1.1.0.
- Major goes up when a change is so big that old parts stop working. Example: 1.1.0 to 2.0.0

We write the version in the README file and in the CHANGELOG file with a short list of what changed. We also make a Git tag for each version, like v1.0.1. Our first version is v1.0.0, our first fix is v1.0.1, and the backend update will be v1.1.0.

Rollback Procedure

A rollback means going back to the last version that worked. We do this when a new update causes a big problem and we cannot fix it in 30 minutes.

- A member tells the group chat that we are rolling back.
- We find the last good version in the Git tags, for example v1.0.0.
- The fastest way is to open Vercel, go to Deployments, and make the older deployment live again. We can also use git revert to undo the bad commit. We never delete history.

- We open the live site and test ordering and the queue.
- We write an issue about what went wrong, fix it on a branch, and release it later as a new patch version.